

2 回の $(n \text{ ビット}) \times (n \text{ ビット})$ 乗算による非再帰型 $n \text{ ビット}$ 擬似乱数の生成 ($n = 64, 128, 192, \dots, 16384$)

(2025 年秋学会アブストラクト再編集)

1 非再帰型 $n \text{ ビット}$ 擬似乱数の生成と検定 ($n = 64, 128, 192, \dots, 16384$)

以下の (S0)-(S5) からなるアルゴリズムで乱数列を生成する：

(S0) $n \text{ ビット}$ の 2 整数 X, Y を準備する。 X, Y の最上位ビット (MSB, Most Significant Bit) は 1 とする。

(S1) 2 数 X, Y を掛けて、 $2n \text{ ビット}$ の数 XY を作る。

(S2) XY のちょうど真ん中にある連続する $n \text{ ビット}$ を取り出して m とおく。

(S3) m の MSB を 1 にしたものを M とし、 M を 2 乗して $2n \text{ ビット}$ の数 MM を作る。

(S4) MM のちょうど真ん中にある連続する $n \text{ ビット}$ を取り出して 1 つの乱数値 z とする。

(S5) (S0) の X, Y を $(X_0, Y_0), (X_1, Y_1), (X_2, Y_2), \dots$ と色々変化させて、各 (X_i, Y_i) ごとに (S0) から (S4) を行なって、多くの乱数値 $z_0, z_1, z_2, z_3, \dots$ を得る。(非再帰型の乱数生成) ($n=64, 128, 192, 256, 384, 512, 1024, 2048, 4096, 8192, 16384$)

[以下、引用を簡単にするため、このアルゴリズムを **MmS** (Multiply and middle-Square, 乗算採中二乗) アルゴリズムと呼ぶことにする]

註 1： (S4) の z を (S3) の m と読み替えて、 (S3)-(S4) を任意回繰返すこともできる。

(S5) の (X_i, Y_i) は以下の様にとる。 まず $e = 2.7182\dots$ を自然対数の底とし、

$(1 + \frac{e}{10}) \times 8 = 10.1746254\dots = (16 \text{ 進表記}) 0xa.2cb4411 \text{ ba}257552 \text{ 232c1139 } \dots$
の左から $n \text{ ビット}$ を B とする。次に 64 ビット の 4 数 p, q, r, s を

$$\begin{aligned} p &= 0x7fffffffffffffe7, & q &= 0x7fffffffffffff5b, \\ r &= 0x39f750241c2d5d33, & s &= 0x32f50fee9b2a32bb \end{aligned}$$

で定め (これらはどれも素数で MSB は 0), P_i, Q_i を以下で決まる 64 ビット 数とする：

$$P_i = r \times i \bmod p, \quad Q_i = s \times i \bmod q.$$

この状況下で、 $X_i[Y_i]$ を、 $n \text{ ビット}$ の B と、 64 ビット の $P_i[Q_i]$ との XOR (排他的論理和、 $\oplus$) を各ビット列の丁度真ん中位置に合わせて採ったものとする。例えば $n = 64$ の場合、 $i = 3$ のときは、 $B = 0xa2cb4411ba257552, P_3 = 0x2de5f06c548817b2$ であるので、 X_3 は以下の様になる：

$$X_3 = B \oplus P_3 = (a2cb4411:ba257552) \oplus (2de5f06c:548817b2) = 8f2eb47d:eead62e0.$$

註 2： P_i が分かっているとき、 P_{i+1} は $P_i + r$ もしくは $P_i + r - p$ なので、簡単に計算できる。 Q_i についても同様である。

註3： MmS における定数 B, p, q, r, s は可変である。ここでは $n = 8192, 16384$ の場合に、 B として $\pi - 2.5 = 0.64159... = 0x0.a43f6a...$ の上位 n ビットを使っている。

〔 n ビット数列 $\{z_i\}_i$ の乱数性の検定〕は、L'Ecuyer & Simard による乱数性テスト集 TestU01 により行う。 $n = 64, 128, \dots, 4096$ の場合は Battery Crush を、 $n = 8192, 16384$ の場合は検定時間との兼ねいで Battery SmallCrush を適用する。Crush には 96 のテスト項目があり、テスト結果として 144 個の p 値を生成する。検定の対象は乱数列 $\{z_i\}_i$ に対して、〔検定対象 a〕 z_i の n ビット全部、〔検定対象 b〕 z_i の最初の 64 ビットのみとする。結果は、全ての n について〔検定対象 a〕,〔検定対象 b〕とも合格である。

註4（乱数生成の高速化）： $X_i[Y_i]$ を作る際、 B と $P_i[Q_i]$ の XOR を、 B の 64 ビット毎全てについて採ると、MmS の(S3)を省略しても（ $z = m$ とする）上記テストをパスする乱数列が生成される。この方法では $(n\text{-bit}) \times (n\text{-bit})$ の乗算が 1 回で済むので、乱数の生成速度が速くなる。（§3 表中の $n = 64$, (S2) m_i の項参照）

2 乱数生成アルゴリズムの $[1, 2) \times [1, 2)$ 上の関数による記述

k を非負の整数とし $[1, 2) \times [1, 2)$ 上の関数 $T_{2^k}(X, Y) = 2^k XY - [2^k XY] + 1$ を考える。（ $[a]$ は a の整数部分を表す。）（記号 X, Y は §1 の X, Y と敢えて混用している。） X, Y として n ビットの $X = 1.u_1 \cdots u_{n-1}$ と $Y = 1.v_1 \cdots v_{n-1}$ を使うとすれば、乗算 XY の結果は $2n$ ビットの $XY \equiv w_0 w_1 \cdots w_{2n-1}$ となる。（ $XY \in [1, 2)$ のとき $w_0 w_1 = 01$, $XY \in [2, 4)$ のとき $w_0 = 1$ 。）このとき、 $2^k XY = w_0 w_1 \cdots w_{k+1} \cdot w_{k+2} \cdots w_{2n-1}$ であり、 $T_{2^k}(X, Y)$ では整数部分の $w_0 \cdots w_{k+1}$ が 1 となるので $T_{2^k}(X, Y) = 1.w_{k+2} \cdots w_{2n-1}$ となる（ $T_{2^k}(X, Y)$ では、 $w_0 \cdots w_{2n-1}$ の頭の $k+1$ ビット $w_0 \cdots w_k$ が落とされる）。

通常計算機では数値に割り当てるビット数は予め決めてあるので（例えば整数で n ビット、その乗算結果は $2n$ ビット）、 $2n - k - 1$ ビットの $T_{2^k}(X, Y)$ は次の乗算の変数として使うために n ビットに切り詰められる、すなわち、

$$T_{2^k}(X, Y) = 1.w_{k+2}w_{k+3} \cdots w_{2n-1} \implies 1.w_{k+2}w_{k+3} \cdots w_{k+n} \quad (n \text{ ビットに切り詰め})。$$
上式で特に $k = (n/2) - 1$ として、式を書き改めると

$$T_{2^{(n/2)-1}}(X, Y) = 1.w_{(n/2)+1}w_{(n/2)+2} \cdots w_{2n-1} \implies 1.w_{(n/2)+1}w_{(n/2)+2} \cdots w_{(3n/2)-1} \quad (n \text{ ビット})$$
となる。 n ビット変数 X, Y における上述の $k = (n/2) - 1$ に対する計算機上の一連の計算を、小数点を取り除いて眺めてみると、2 数 X, Y を掛けて、その結果の $2n$ ビットから、真ん中の n ビット $w_{n/2}w_{(n/2)+1} \cdots w_{(3n/2)-1}$ を取り出して（＝頭の $n/2$ ビットと尻尾の $n/2$ ビットを落として）、その MSB である $w_{n/2}$ を 1 にしていることが分かる。これらは §1 の MmS アルゴリズムの (S0) から (S3) の M を得るところまでに対応している。(S3) の後半から (S4) までは、関数値 $T_{2^{(n/2)-1}}(M, M)$, $M \in [1, 2)$, から尻尾の $n/2$ ビットをカットして、 n ビットを取り出すことが対応している（この時は MSB を 1 にしないでそのまま使う）。以上のように関数値のビット長が n ビットに制限された $[1, 2) \times [1, 2)$ 上の $T_{2^k}(X, Y)$ の計算を、便宜上、「ビット長固定型」の $T_{2^k}(X, Y)$ 計算と

呼ぶことにすれば、アルゴリズム MmS は、2つの関数 $T_{2^{(n/2)-1}}(X, Y)$, $T_{2^{(n/2)-1}}(M, M)$ のビット長固定型計算により記述されるということになる。

3 「ビット長非固定型」の $T_{2^{(n/2)-1}}(X, Y)$ 計算と乱数性

§2 で述べた $T_{2^k}(X, Y)$ の「ビット長固定型」の計算では、 n ビットより長い乗算結果を n ビットに丸めるので、数理的に扱いにくい。そこで丸め操作を全て省略した本来の $T_{2^k}(X, Y)$ の計算（ビット長非固定型の計算）を考える。関数 $T_{2^{(n/2)-1}}$ の「ビット長非固定型」計算によって得られる（乱）数列 $\{\zeta_i\}_i$ と、「ビット長固定型」の計算で得られる乱数 $\{z_i\}_i$ の乱数性を検定により比べてみる。検定は、TestU01(Cruch) を $n = 64, 128, \dots, 512$ の [検定対象 b] に適用した。

採中二乗 [検定対象 b]	$n = 64$		$n = 128$		$n = 192$		$n = 256$		$n = 384$		$n = 512$	
	固定長	非固定	固定長	非固定	固定長	非固定	固定長	非固定	固定長	非固定	固定長	非固定
(S2) m_i	o	←	o	←	↓ 2	←	↓ 1	←	↓ 6	←	↓ 5	←
(S4) z_i	o	o	o	o	o	o	o	o	o	o	o	o

表中の (S2) m_i , (S4) z_i は、それぞれ (S2), (S4) の（乱）数列 $\{m_i\}_i$, $\{z_i\}_i$ についての検定結果を表す。← は同一の検定となることを表す。また、 $\downarrow e$ (該当なし), $\downarrow e$ は、それぞれ p 値が 10^{-15} 以下, 10^{-300} 以下になったテストの項目数が e であることを表す。o は全ての p 値が $\downarrow, \downarrow$ に該当しないことを表す。表から、「ビット長非固定型」の乱数列 $\{\zeta_i\}_i$ と「ビット長固定型」の乱数列 $\{z_i\}_i$ の乱数性は同様であると考えられる。なお、註4の方法（(S3) を省略する）は表中の (S2) m_i , $n = 64$ の結果 'o' から考案された。

4 $T_{2^k}(X, X)$ におけるシフトの量と乱数性の関係

MmS アルゴリズムの数理的解析を試みるため、(S0) の $T_{2^k}(X, Y)$ を $T_{2^k}(X, X)$ と単純化して考える、すなわち、 $T_{2^k}(X, X) = 2^k X^2 - \lfloor 2^k X^2 \rfloor + 1$ である。 $T_{2^k}(X, X)$ で k の値を $k = (n/s) - 1$, $s = 4, 8, 16$, と変化させた場合について、 $T_{2^k}^\ell$ の ℓ の値（ τ_{mul} で表す）を変化させた時の乱数性の挙動を調べてみる。（ τ_{mul} は、(S1) と (S3) で行われる乗算の総回数に相当する（註1参照）。）（註： $T_{2^{(n/s)-1}}$ では乗算結果の $2n$ ビットから、頭の n/s ビットが落とされる。）

$T_{2^k}(X, X)$	$k = (n/4) - 1$					$k = (n/8) - 1$					$k = (n/16) - 1$				
$n \Rightarrow$	256	384	512	1024		256	384	512	1024		256	384	512	1024	
XORpos $\Rightarrow$	C	C	C	C	T	C	C	C	C	T	C	C	C	C	T
$\tau_{\text{mul}} = 2$	3	4	4	4	-	-	-	-	-	-	-	-	-	-	-
$\tau_{\text{mul}} = 3$	0	0	0	0	-	4	4	4	-	-	-	-	-	-	-
$\tau_{\text{mul}} = 4$	-	-	-	-	4	0	0	3	4	-	-	-	-	-	-
$\tau_{\text{mul}} = 5$	-	-	-	-	0	-	-	0	0	-	4	-	-	-	-
$\tau_{\text{mul}} = 6$	-	-	-	-	-	-	-	-	-	-	3	4	-	-	-
$\tau_{\text{mul}} = 7$	-	-	-	-	-	-	-	-	-	4	0	3	4	4	-
$\tau_{\text{mul}} = 8$	-	-	-	-	-	-	-	-	-	0	-	0	0	3	-
$\tau_{\text{mul}} = 9$	-	-	-	-	-	-	-	-	-	-	-	-	-	0	-
$\tau_{\text{mul}} = 15$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	4
$\tau_{\text{mul}} = 16$	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0

上表は、 $n = 256, 398, 512, 1024$ の場合において、 $T_{2^{(n/s)-1}}(X, X)$ のビット長固定型

計算を $\tau_{\text{mul}} = 2, 3, 4, \dots, 16$ に対して行い, TestU01 ([検定対象 b]) を適用した結果である。(註: 表中 XORpos の $C[T]$ は, B と P_i から X_i を作る際, XOR を B の中央 (Center) [最後尾 (Tail)] で採った場合の結果である。) 検定を全てのテストについて行うことは大変なので, § 3 の表中で p 値が極端に悪かったテスト項目番号

#11(BirthdaySpacings), #31(Gap), #37(Permutation), #55(SumCollector)

の4つに絞って行った。表中の数値は, p 値が 10^{-15} 以下になったテスト項目の数 (0 ~ 4) である。表より以下の事が見てとれる:

(O₁) $k = (n/s) - 1$, $s = 4, 8, 16$, において, $\tau_{\text{mul}} = s/2$ [$n=1024T$ では $\tau_{\text{mul}} = s$] 前後で値が 0 になってくる,

(O₂) 値 $\tau_{\text{mul}} = s/2$ [$\tau_{\text{mul}} = s$] は n に依存しない, さらに,

(O₃) $T_{2(n/s)-1}(X, X)$ では乗算結果の先頭 n/s ビットが落とされるので, $\tau_{\text{mul}} = s/2$ [$\tau_{\text{mul}} = s$] では, 延べ $n/2$ [n] ビットが落とされる (s に依存しない), また,

(O₄) $n=1024C$ と $1024T$ の比較から, 乱数性が現れる τ_{mul} は, B から X_i を作る際に XOR が採られるビット位置 (ほぼ $n/2$ [n]) に強く依存している。

5 数理的な考察

§4 の (O₁)~(O₄) について数理的考察を行う。記述を見やすくするため,

$$\tilde{T}_\ell(X) \equiv T_{2^k}^\ell(X, X), \ell = 1, 2, \dots, \tilde{T}_0(X) = X$$

とおく。整数値関数 $J_\ell(X)$ を,

$$\tilde{T}_\ell(X) = 2^k(\tilde{T}_{\ell-1}(X))^2 - J_\ell(X), J_\ell(X) \equiv \lfloor 2^k(\tilde{T}_{\ell-1}(X))^2 \rfloor - 1$$

により定める。値 $\Delta(\ll 1)$ に対して, $\tilde{T}_\ell(X + \Delta)$, $J_\ell(X + \Delta)$ も同様に定める。

$\tilde{J}_\ell(X, \Delta) \equiv J_\ell(X + \Delta) - J_\ell(X)$ とおく。このとき,

$W_0(X, \Delta) = \Delta$, $U_0(X, \Delta) = 0$, から始めて, $\ell = 1, 2, \dots$ に対して, 帰納的に

$$W_\ell(X, \Delta) = 2^k W_{\ell-1}(X, \Delta) \{2\tilde{T}_{\ell-1}(X) + W_{\ell-1}(X, \Delta)\} \quad (\text{式 1}),$$

$$U_\ell(X, \Delta) = 2^k U_{\ell-1}(X, \Delta) \{2\tilde{T}_{\ell-1}(X) + 2W_{\ell-1}(X, \Delta) + U_{\ell-1}(X, \Delta)\} - \tilde{J}_\ell(X, \Delta)$$

と定めれば,

$$\tilde{T}_\ell(X + \Delta) = \tilde{T}_\ell(X) + W_\ell(X, \Delta) + U_\ell(X, \Delta) \quad (\text{式 2})$$

が成り立つ。

$\tilde{T}_\ell(X)$ の局所的な挙動を調べるため $\tilde{T}_\ell(X)$ から $\tilde{T}_\ell(X + \Delta)$ までの全ての値が, $\tilde{T}_\ell(X)$ のグラフの同一線上にあると仮定する。このとき $\tilde{J}_i(X, \Delta) = 0$ より, (式 2) で $U_\ell(X, \Delta) = 0$ となるので, $\tilde{T}_\ell(X + \Delta) - \tilde{T}_\ell(X) = W_\ell(X, \Delta)$ の ℓ, Δ による依存性を調べることになる。(式 1) より,

$$W_\ell = 2^{\ell(k+1)} W_0 \left\{ \prod_{i=0}^{\ell-1} \tilde{T}_i(X) \right\} \times \left\{ \prod_{i=0}^{\ell-1} \{1 + W_i/2\tilde{T}_i(X)\} \right\} \quad (\text{式 3})$$

である。(式 3) で, $k = (n/s) - 1$ (1 回毎に n/s ビット落とす), $\Delta = 2^{-(n/t)}$, $t = 1, 2$, (XOR を採るビット位置にほぼ対応) の場合を考える。さらに, 各 ℓ に対し, $\theta_\ell \in (0, 1)$ を $\prod_{i=0}^{\ell-1} \tilde{T}_i(X) = (2^{\theta_\ell})^\ell$ を満たす数とする。このとき, (式 3) は $W_\ell = 2^{\ell((n/s)+\theta_\ell)-(n/t)}$ $\left\{ \prod_{i=0}^{\ell-1} \{1 + W_i/2\tilde{T}_i(X)\} \right\}$ となる。この式で, ℓ を $1, 2, \dots$ と変化させたとき, 右辺の 2 の累乗の指数の符号が − (負) から + (正) に変わる辺りから, 乱数性が顕著に出現す

ると考えられる。不等式

$$\ell((n/s) + \theta_\ell) - (n/t) \geq 0 \quad (\Leftrightarrow \ell \geq \frac{s}{t} \cdot (1 - \frac{\theta_\ell s}{n + \theta_\ell s}))$$

を満たす最小の ℓ は $\frac{s}{n} \ll 1$ のとき $\frac{s}{t}$ である。 τ_{mul} として $\check{\ell} \equiv \frac{s}{t}$ をとると $\frac{n}{s} \times \check{\ell} = \frac{n}{t}$ となる。 $\check{\ell} = \frac{s}{t}$ から §4 の (O₁), (O₂), (O₄) が, $\frac{n}{s} \times \check{\ell} = \frac{n}{t}$ から (O₃) が説明される。 $T_{2^k}(X, Y)$ の場合は, $T_{2^k}(X + \Delta_X, Y + \Delta_Y)$, $\Delta_X, \Delta_Y = 2^{-(n/t)}$, $t = 1, 2$, と仮定して (この時 $(X_i, Y_i)_i$ の定め方が効いてくる) 同様の計算を行うと, 最終的に $\Delta = \max\{\Delta_X, \Delta_Y\}$ が効いてくることが分かる。 §1 の MmS は $s = 2, t = 2$ であり, $\check{\ell} = \frac{s}{t} = \frac{2}{2} = 1$ となる (§3 表中の $n = 64, 128$ における (S2) m_i の結果 "o" はこれに当てはまる)。 MmS の τ_{mul} は 2 ((S1) と (S3)) で $\check{\ell} = 1$ より大きく ($\check{\ell}$ は n に依存しない), MmS のアルゴリズムが n によらないことが説明される。

〔補足 1〕 ($T_{2^k}(X, Y)$ を用いた再帰型乱数生成) $Z_0 = B$ から始めて,

$$Z_i = T_{2^{(n/2)-1}}(Z_{i-1}, Z_{i-1} \oplus (i-1)), \quad i = 1, 2, \dots,$$

についてビット長固定型計算を行うと, von Neumann 型 (平方採中法型) の再帰型乱数生成を行うことができる。 ($\oplus(i-1)$ は, 生成される乱数列がループに入るのを防ぐために挿入されている。) この方法により生成される乱数列 $\{z_i\}$ は (z_i として Z_i の計算において MSB を 1 にする前の値 m_i を使う, 即ち $z_i = m_i$ ((S2) 参照)), Crush テストをパスする。